AS YOU SIT DOWN

Get your workspace open

FOUR THINGS

- VS Code open on your **ECON209** folder
- A new notebook in **labs/code/** — **lab2.ipynb**
- Your AI assistant signed in beside it
- The **Data Wrangler** extension installed — Extensions panel, search `Data Wrangler`

IF SOMETHING ISN'T WORKING

- Sit next to someone whose setup runs
- Follow the session now, fix yours after
- Tonight is **mostly you driving**. I am here for when it stalls

APPLIED ECONOMETRICS · RICE UNIVERSITY

Lab Two

Get the data in. Look at it properly. Then break it on purpose, and find out which kinds of broken actually matter.

Jessica Rood
Teaching Assistant

jr120@rice.edu
Questions, any time

Sewall 207
Mondays

TONIGHT

Two things, and most of it is yours

1. Load it and look at it. Get `public.dat` into pandas, then explore in Data Wrangler — chasing *your* questions, not mine
2. Break it on purpose. Add random error a hundred times and watch what happens to the answer

THE PATTERN, ALL NIGHT

- **You ask.** Say what you want, not how to do it
- **It drafts.** Fast, confident, usually right
- **You check.** Against something you knew beforehand

The third step is the only one that is yours.

PART ONE

Get it in

- davidcard.berkeley.edu/data_sets.html
- Unzip **njmin.zip** into **`labs/data/`**, then open **read.me** and **codebook**

PUBLIC.DAT — NO HEADER, MISSING = A BARE FULL STOP

```
 46 1 0 0 0 0 0 1 0 0   0 30.00 15.00  3.00  .
506 2 1 0 0 0 0 1 0 0   0  3.00  7.00  2.00  .
```

YOU

`public.dat` has no header row. Read `data/codebook`, work out the names and the missing-value marker, and load it into `df`. Explain before you run.

THEN CHECK

410 × 46

- Both numbers came from the read.me and the codebook *before* you asked

Open it in Data Wrangler

TWO WAYS IN

- Run a cell ending in `df`, then click **Open in Data Wrangler** above the output
- Or the **Variables** panel → the grid icon next to `df`

WHAT YOU GET

- Every column with its **type**, **missing count** and distribution — all 46 at once, no code. Filter and sort by clicking

THE PART THAT MATTERS

- Every click **writes the pandas line** for the step you took, and sends it back into your notebook
- So poking around by hand still leaves you **code you can rerun**

Same bargain as the assistant: it types, you decide whether the step was right.

Go find something

PICK A THREAD AND PULL IT

- Which columns are **missing most**, and is that spread evenly across the two states?
- Is **SHEET**, the store id, actually unique?
- Does any **price** or **hours** value look impossible?
- Which variables are **codes**, not quantities? What happens if you average one?

THE ONLY RULE

- When you find something odd, **go back to the codebook** before deciding what it means
- Write one line: **what you found**, and **how you would know if you were wrong**

Starting points, not a worksheet. If you end up somewhere I did not list, that is the good outcome. Bring one thing back to the room.

Break it on purpose

YOU

Wave-1 soda price by state — mean, sd, and the difference between them.

```
        mean      sd     n
PA     0.975   0.069    77
NJ     1.062   0.085   325
--------------------------
NJ - PA   +0.087
```

Every variable has its own **n** — check it, do not assume it.

TWO WAYS TO BE WRONG

- **You measured it badly.** The price you wrote down is off
- **You classified it badly.** Stores are filed under the wrong state

Nearly a full sd apart. Ask for the **difference** itself, not two columns to compare by eye. PS1 III runs this same pair on **FTE employment**; tonight, price.

A dime either way on the price

```
rng = np.random.default_rng(209)
gaps = []
for i in range(100):                  # 100 passes
    e1 = rng.uniform(-.10, .10, n)    # a dime either way
    psoda1 = d.PSODA + e1             # a new variable
    m = psoda1.groupby(d.STATE).agg(["mean","std"])
    gaps.append(m.loc[1,"mean"] - m.loc[0,"mean"])
```

Store the **gaps**, not the two columns — a hundred answers to the one question you asked.

THE DIFFERENCE HELD

	was	now
PA mean	0.975	0.975
NJ mean	1.062	1.062
NJ - PA	+0.087	+0.087
PA sd	0.069	**0.091**
NJ sd	0.085	**0.102**

All 100 differences sat between **+0.068 and +0.112**; none negative. Only the **sd** moved — by 20–30%.

Now put the stores in the wrong state

```
rng = np.random.default_rng(209)
gaps = []
for i in range(100):
    e1 = rng.uniform(-.10, .10, n)   # the same e1
    NJm1 = np.where(e1 > 0, d.STATE, 1 - d.STATE)
    m = d.PSODA.groupby(NJm1).agg(["mean","std"])
    gaps.append(m.loc[1,"mean"] - m.loc[0,"mean"])
```

Same loop, same draw — but **e1>0** half the time, so each pass mislabels **about half the sample**, a different half each time.

THE DIFFERENCE DID NOT

	was	now
PA mean	0.975	1.045
NJ mean	1.062	1.045
NJ - PA	+0.087	-0.000
PA sd	0.069	0.088
NJ sd	0.085	0.089

Both “states” collapse onto the pooled mean of **1.045**, and **49 of 100 differences came out the wrong sign** — a coin flip.

THE THING TO TAKE AWAY

Each error breaks one number

ERROR IN WHAT YOU MEASURE

+ Lands in the **standard deviation**. The means hold
+ Wrong both ways, so it averages out
+ The comparison survives

ERROR IN HOW YOU CLASSIFY

- Lands in the **means**. The groups blur together
- Always toward zero — here, all the way
- The comparison is gone

The problem set asks for **one** draw of `e1`. A hundred is how you know that draw was not a fluke.

BEFORE YOU GO

Three things

- [ ] Save `lab2.ipynb` in `labs/code/`, cells run and output showing — including whatever *you* went chasing.
- [ ] Ask your assistant to add tonight to `decisionlog.md` — what you did, and **why**. Then read what it wrote.
- [ ] **Problem Set 1 is posted.** Getting this file loaded and checked is the part that stops most people.

Next week: Git, GitHub, and a clinic on Problem Set 1. Bring whatever is not working.

`jr120@rice.edu`